



A Multitask Training Approach to Enhance Whisper with Open-Vocabulary Keyword Spotting

Yuang Li, Min Zhang, Chang Su, Yinglu Li, Xiaosong Qiao, Mengxin Ren,
Miaomiao Ma, Daimeng Wei, Shimin Tao, Hao Yang

Huawei Translation Services Center, China

{liyuang3, zhangmin186, suchang8, liyinglu, qiaoxiaosong, renmengxin2, mamiaomiao, weidaimeng, taoshimin, yanghao30}@huawei.com

Abstract

The recognition of rare named entities, such as personal names and terminologies, is challenging for automatic speech recognition (ASR) systems, especially when they are not frequently observed in the training data. In this paper, we introduce keyword spotting enhanced Whisper (KWS-Whisper), a novel ASR system that leverages the Whisper model and performs open-vocabulary keyword spotting (OV-KWS) on the hidden states of the Whisper encoder to recognize user-defined named entities. These entities serve as prompts for the Whisper decoder. To optimize the model, we propose a multitask training approach that learns OV-KWS and contextual-ASR tasks. We evaluate our approach on Chinese Aishell hot word subsets and two internal code-switching test sets and show that it significantly improves the entity recall compared to the original Whisper model. Moreover, we demonstrate that the OV-KWS can be a plug-and-play module to enhance the ASR error correction methods and frozen Whisper models.

Index Terms: speech recognition, keyword spotting, prompt

1. Introduction

Recent years have witnessed the rise of end-to-end (E2E) automatic speech recognition (ASR) models [1, 2, 3] due to their simplicity and unified architecture. However, these models struggle to recognize proper nouns that infrequently occur in the training data. A common approach is to construct a list of hot words and apply it to shallow fusion [4, 5] or deep fusion [6, 7, 8, 9, 10]. Shallow fusion enhances the hot words' score during beam search decoding, but it requires tuning the optimal fusion weight. The deep fusion method trains a contextual encoder jointly with the ASR model from scratch. These methods map the entity words and the speech signal into the same feature space, and the decoder leverages both contextual and acoustic information to generate the transcription. To adapt existing ASR models, some methods use adaptors [11, 12] to modify the intermediate features of the ASR model or pointer networks [13] to revise the output distributions. Moreover, the shallow and deep fusion methods can be combined to enhance the performance [14, 15].

Whisper [16] stands as a cutting-edge ASR model. It leverages the Transformer architecture [17] and was trained on an extensive dataset comprising 680k hours of speech data. Unlike its predecessors, which necessitate a separate biasing module, Whisper dynamically adjusts its output by incorporating a straightforward prompt during decoding. In this paper, we introduce an innovative technique called KWS-Whisper (Keyword Spotting Enhanced Whisper). This method integrates an open-vocabulary keyword spotting (OV-KWS) module between the encoder and decoder of Whisper. The OV-KWS module

draws inspiration from vision-based keyword spotting methods [18, 19] and constructs a cosine similarity matrix using features from the Whisper encoder. To enhance Whisper's comprehension of prompts, we employ a multitask training approach that combines OV-KWS and contextual-ASR tasks. Our system enhances entity recall with absolute improvements of up to 80% on Aishell hot word subsets [20] and up to 10% on internal code-switching datasets. However, we noticed that the KWS-Whisper fine-tuned on a small-scale dataset has higher mixed error rates (MERs) than the Whisper model on real-world internal datasets due to catastrophic forgetting. Therefore, we propose to use OV-KWS as an independent module for frozen Whisper models. We conducted experiments for various scenarios: 1) ASR error correction based on Pinyin or the large language model (LLM); 2) directly prompting the Whisper decoder with spoken-form prompts that resemble the historical transcription. The results show that our system can achieve significant MER reductions of 2.4%, 1.4%, and 1.8% on internal datasets for Whisper-small, medium, and large models respectively. Compared to our previous work [21], the lightweight OV-KWS module and the multitask training approach markedly enhance the system's adaptability, efficiency, and accuracy.

2. Methodology

2.1. System overview

A database is first constructed by extracting acoustic representations for pre-defined entity words. As shown in Figure 1 (a), a text-to-speech (TTS) model followed by the Whisper encoder is used to extract hidden states, which are later matched with the input utterance features. As illustrated in Figure 1 (b), the proposed KWS-Whisper identifies entity words before decoding by leveraging encoder hidden states. Specifically, we aggregate hidden representations from all Whisper encoder layers, weighted by learnable coefficients. A cosine similarity matrix is then computed between the input utterance features and stored entity word features. A binary Convolutional Neural Network (CNN) detects the presence of an entity word by recognizing the diagonal pattern in the similarity matrix (Figure 1 (c)). Finally, predicted entity words guide the Whisper decoder for improved recognition accuracy.

2.2. Open-vocabulary keyword spotting

OV-KWS is framed as a binary classification task. The model predicts the presence of an entity word within an utterance by analyzing the cosine similarity matrix between the hidden states of entity words and the input utterance. Notably, if an entity word exists in the utterance, a diagonal pattern emerges (Figure 1 (c)). Given that this is a local pattern, we opt for a CNN as

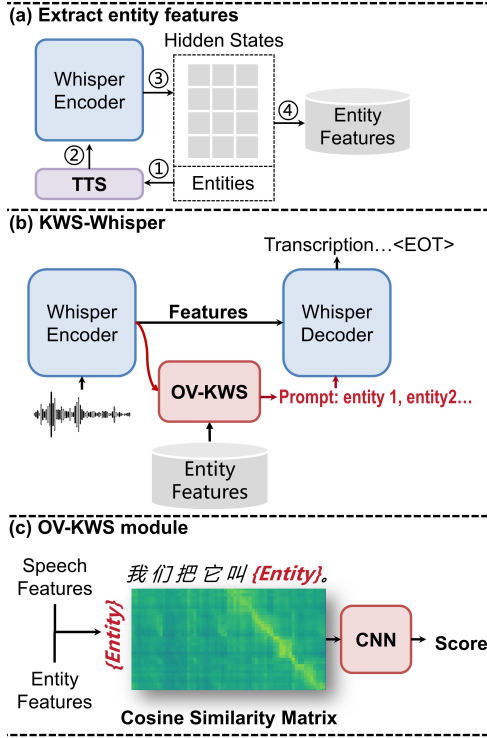


Figure 1: (a) Entity Features are extracted using TTS followed by Whisper encoder. (b) Flowchart of the KWS-Whisper model. (c) Illustration of the OV-KWS module.

our classification model. To extract features, we follow a common approach of using the weighted sum of multi-layer features from the Whisper encoder [22]. The OV-KWS module exhibits linear computational complexity to the number of keywords. For higher efficiency, we employ an ultra-lightweight CNN comprising only four layers and 0.2 million parameters. Additionally, we leverage GPU parallelization to batch and execute multiple keywords simultaneously. An easy way to construct the dataset for the CNN classifier is to select positive words according to the transcription and choose negative words randomly. However, this approach is too simplistic, as the negative samples may have significant differences from the positive samples, resulting in overfitting. As such, we adopt hard negative samples [23], which are more likely to have overlaps with positive samples, increasing the difficulty of the classification task.

2.3. Multitask fine-tuning

The CNN classifier’s capacity is limited by its small size. Therefore, it is necessary to fine-tune the Whisper model for the OV-KWS task while preserving its ASR ability. To this end, we propose to fine-tune all parameters of the Whisper model on both the OV-KWS task and the contextual-ASR task jointly with the loss function shown in Equation 1. During fine-tuning, we incorporate words from the ground-truth transcription into the prompt following the naive prompt format in Table 1. This enables the Whisper decoder to better understand the prompt format and leverage the word in the prompt.

$$Loss = \mathcal{L}_{ASR} + \alpha \times \mathcal{L}_{KWS} \quad (1)$$

Table 1: Prompts for the Whisper decoder.

naive prompt (translation)	实体1, 实体2, 实体3 entity 1, entity 2, entity 3
spoken-form prompt (translation)	今天演讲的主题是这个呃, 实体1、实体2、实体3。好, 那我就继续讲。 The topic of today’s speech is, ah, entity 1, entity 2, entity 3. Okay, then I’ll continue.

where the loss is the weighted sum of ASR loss $\mathcal{L}_{ASR}$ and KWS classification loss $\mathcal{L}_{KWS}$. The weight α was set to three during training.

2.4. Prompting Whisper decoder

During training, the Whisper decoder incorporates contextual information by using the transcription of the previous utterance as a prompt. In inference, any relevant text-only prompt can serve as the historical context for decoding. For multitask training, we adopt a straightforward prompt format (i.e., concatenating all entity words) which performs remarkably well on in-domain test sets (Aishell). However, this format can lead to hallucinations in more complex spontaneous conversations involving code-switching. Additionally, the fine-tuned model’s ASR performance may slightly degrade due to its small-scale training dataset. Therefore, we explore using OV-KWS as a plug-in for correcting ASR errors and prompting frozen Whisper models.

2.5. OV-KWS plug-in for frozen Whisper models

We can use the OV-KWS plug-in for:

- **ASR error correction:** 1) Pinyin-based method: We segment each transcription into words and replace the Chinese word if it has the same Pinyin (pronunciation) as the entity word. For English words, we compute the edit distance and replace the word if the distance is below a pre-defined threshold. 2) LLM-based method: We use ChatGLM3-6B [24] and the prompt in Table 2. In our experiments, we provide two examples for the LLM.
- **Prompting Whisper decoder:** Since the Whisper models are not fine-tuned, we use spoken-form prompt (Table 1) to prevent hallucinations which incorporates disfluencies and indicate that the subsequent speech was a spontaneous talk. Compared to the naive prompt, the spoken-form prompt has a more similar form to the historical transcriptions used during the training of the Whisper model.

Table 2: The prompt for LLM-based ASR error correction. The translation of the Chinese prompt is given in parentheses with italics.

User: 请根据可能出现的关键词修改ASR文本。 (Please modify the ASR transcription according to possible keywords.) 关键词 (keywords): {word1}, {word2}。 识别文本 (recognized text): {ASR transcription}
Response: 修改文本 (modified text):

3. Experimental Setups

3.1. Training configurations

We selected the Whisper-small model for the multitask training. In subsequent experiments, we also compared the Whisper-small, medium, and large-v2 versions without fine-tuning. For the CNN, we employed a lightweight four-layer architecture. Each layer had 128, 128, 256, and 256 channels respectively, and a kernel size of three. In the first training phase, we froze the Whisper encoder and trained the CNN classifier for OV-KWS and the Whisper decoder for ASR, for 10 epochs, with a batch size of 128 and a learning rate of 5×10^{-5} . The total number of trainable parameters was 153.8 million (CNN 0.2 million + decoder 153.6 million). In the second training phase, we unfroze the Whisper encoder and trained the entire model for another 50 epochs, with a learning rate of 3×10^{-5} . The number of trainable parameters increased to 241.9 million. Note that for the ASR task, we applied a random concatenation strategy [25] to increase the length of the utterance. We used a beam size of five for decoding.

3.2. Datasets

To construct the training set for OV-KWS and ASR, we utilized three datasets: Aishell-1 [26], LibriSpeech [27], and TALCS [28]. Aishell-1 is a Mandarin ASR dataset with 150 hours of speech recordings. LibriSpeech is an English ASR dataset, from which we selected the train-clean subset containing 100 hours of speech recordings. TALCS is a Mandarin-English code-switching dataset with 578 hours of speech data from English teaching scenes. For each utterance in these datasets, we randomly sampled positive words and generated speech using edge-tts¹. We compared the performance of the model trained with only Chinese data (zh) and the model trained with the mixture of three datasets (zh + en).

Table 3: Statistics of ASR test sets.

Dataset	Utterances	Duration (min)	Entities
Aishell-dev	1334	113	371
Aishell-test	808	76	226
Internal-1	99	41	150
Internal-2	112	47	346

We evaluated the performance of OV-KWS and ASR on four different test sets, which consisted of two internal datasets and two open-source datasets. The internal datasets contain technical talks and manually labeled entity words that are mainly in Chinese but also include some English terms. The open-source datasets are the Aishell hot word subsets [20] in Chinese. The statistics of these datasets are shown in Table 3.

3.3. Evaluation metrics

We evaluated the performance of OV-KWS using the F1 score, which is the harmonic mean of the precision and recall. For ASR, we adopted two metrics: mixed error rate (MER) and entity recall. MER is a modified version of character error rate (CER) that can handle code-switching situations between English and Chinese. In this metric, we treat each Chinese character and each English word as a single unit. Therefore, MER is

¹<https://github.com/rany2/edge-tts>

equivalent to CER for Chinese and word error rate (WER) for English.

4. Experimental Results

4.1. Results for open-vocabulary keyword spotting

We evaluated the performance of the CNN classifier for OV-KWS using the F1 score. Table 4 shows the results of different training settings. We first trained the model with a frozen Whisper encoder on the Aishell dataset only and obtained F1 scores of around 0.6 on the Aishell test sets and 0.68 on the internal datasets. Then, we evaluated the model trained with a mixture of three datasets and observed consistent improvement in the F1 scores. Finally, we unfroze the Whisper encoder, which increased the number of parameters of the OV-KWS task from 0.2 million to 88.4 million and achieved significant improvement in the F1 scores. The model trained with the Aishell dataset performed slightly better on the Aishell-dev and Aishell-test sets, with F1 scores of 0.85 and 0.89, respectively. On the internal datasets, the model trained with the mixed dataset achieved higher F1 scores, reaching around 0.9. The comparison between single-task training and multitask training is presented in Table 5. The results indicate that simultaneous optimization of ASR and OV-KWS does not adversely affect the performance of the OV-KWS task instead, it leads to a higher F1 score on the Internal-1 dataset. Figure 2 illustrates the weight assigned to each layer of the Whisper encoder. The hidden states from the 8th to the 12th layers have a greater impact on the OV-KWS task. Moreover, the weight for the deeper layers increased slightly after fine-tuning, suggesting that the Whisper encoder is more adapted to the OV-KWS task.

Table 4: The performance of the OV-KWS module measured by **F1 score**. We compared the models trained with Chinese-only data (zh) and Chinese-English-code-switching data (zh + en). We also compared the results with the Whisper model frozen or unfrozen.

Training data	zh		zh + en	
	×	✓	×	✓
Unfreeze Whisper				
Aishell-dev	0.62	0.85	0.64	0.83
Aishell-test	0.60	0.89	0.61	0.84
Internal-1	0.68	0.75	0.71	0.90
Internal-2	0.68	0.82	0.72	0.88

Table 5: The comparison between single-task training and multitask training of the OV-KWS model measured by **F1 score**.

	OV-KWS	OV-KWS + ASR
Aishell-dev	0.86	0.83
Aishell-test	0.86	0.84
Internal-1	0.87	0.90
Internal-2	0.88	0.88

4.2. Results for KWS-Whisper

The multitask training of our KWS-Whisper model yielded the ASR performance shown in Table 6. Compared to the original Whisper-small model, the fine-tuned KWS-Whisper model achieved a significantly lower MER and higher entity recall

Table 6: The performance of KWS-Whisper measured by **MER (%) / Entity Recall (%)**. The Whisper-small model was unfrozen and trained jointly with the OV-KWS module. "zh" means the model was fine-tuned on Chinese-only data. "zh + en" represents the model was trained on Chinese-English-code-switching data. We prompted the Whisper decoder with ground-truth entities and predicted entities respectively.

Model Prompt	Whisper-small (baseline)		KWS-Whisper-small (zh)			KWS-Whisper-small (zh + en)		
	×	ground-truth	×	ground-truth	prediction	×	ground-truth	prediction
Aishell-dev	16.7 / 6.3	13.3 / 81.6	10.4 / 20.9	7.2 / 87.1	7.6 / 84.2	12.1 / 19.6	8.5 / 91.5	9.2 / 88.4
Aishell-test	20.1 / 10.1	15.9 / 82.1	11.6 / 14.0	8.3 / 84.4	8.6 / 82.4	13.5 / 9.8	9.7 / 91.6	10.2 / 87.7
Internal-1	5.9 / 78.5	5.3 / 95.3	14.7 / 49.0	13.5 / 76.7	13.9 / 70.2	8.9 / 61.7	8.7 / 86.8	7.7 / 86.3
Internal-2	9.7 / 60.8	8.9 / 93.7	20.0 / 32.5	17.4 / 69.5	17.2 / 64.8	16.6 / 35.9	13.0 / 76.4	13.2 / 71.4

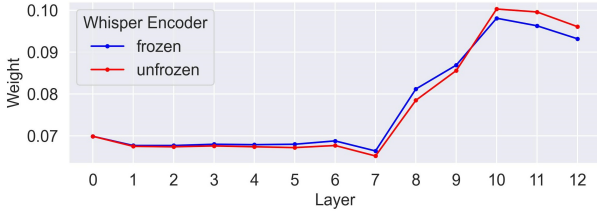


Figure 2: The weights for Whisper encoder layers.

on the Aishell-dev and Aishell-test sets. The model that was trained on the mixed dataset exhibited higher entity recall across all datasets, indicating better prompting function and generalizability. Remarkably, the KWS-Whisper model with predicted entities surpassed the Whisper-small model with ground-truth entities in entity recall, reaching 88.4% and 87.7% on the Aishell-dev and test sets respectively. Furthermore, the KWS-Whisper model outperformed the state-of-the-art SeACo-Paraformer model [20], which has entity recalls of 86% and 87% on the Aishell-dev and Aishell-test sets.

However, we found that though the entity recalls could be enhanced on internal test sets, the MERs increased slightly. This was mainly because we fine-tuned KWS-Whisper on small-scale datasets, and the domain of the training data (audiobook and teaching scenes) differed from that of the test sets (technical talks). As a result, Whisper’s generalizability was reduced. Hence, we suggest using OV-KWS as a plug-in for ASR error correction or applying frozen Whisper models. Table 7 shows that error correction with Chinese Pinyin, using the recognized entity from the OV-KWS model, consistently improved the MER and entity recalls for all Whisper model sizes. The LLM-based correction achieved higher entity recalls than the naive Pinyin-based method, as it incorporated contextual information. However, the MER increased due to the LLM’s hallucinations. We also employed the prompting approach in KWS-Whisper, without fine-tuning various Whisper models. This method outperformed error correction-based methods significantly, as it considered acoustic features. Nevertheless, we discovered that the naive prompt slightly raised the MER compared to the original model. The main reason was that the Whisper model tends to omit filler words and disfluencies if the prompt is in well-formatted text. This problem could be alleviated by using the spoken-form prompt that mimics a presentation transcription. Compared to the naive prompt and no prompt, spoken-form prompts consistently improved the MER. For example, using the OV-KWS model and the spoken-form prompt, the MER decreased from 4.4% to 2.9% and from 5.6% to 3.6% on the Internal-1 and Internal-2 subsets respectively for

Table 7: The performance of various systems with OV-KWS as an independent module measured by **MER (%) / Entity Recall (%)**. We investigated the effects of two ASR error correction techniques: Pinyin-based and LLM-based, and two types of prompts for the Whisper decoder: naive and spoken-form.

Model	Method	Internal-1	Internal-2
Whisper-small	×	5.9 / 78.5	9.7 / 60.8
	Correction-Pinyin	5.7 / 82.9	9.1 / 68.2
	Correction-LLM	6.9 / 85.2	12.7 / 74.3
	Prompt-Naive	5.4 / 94.0	9.0 / 90.2
	Prompt-Spoken	4.0 / 95.9	6.8 / 91.1
Whisper-medium	×	4.8 / 84.2	6.7 / 70.4
	Correction-Pinyin	4.8 / 85.2	6.3 / 78.3
	Correction-LLM	6.1 / 88.9	9.3 / 80.9
	Prompt-Naive	6.8 / 94.3	14.6 / 90.3
	Prompt-Spoken	4.0 / 95.6	4.8 / 94.4
Whisper-large-v2	×	4.4 / 87.3	5.6 / 79.9
	Correction-Pinyin	4.3 / 88.9	5.5 / 83.9
	Correction-LLM	6.0 / 89.9	7.7 / 85.1
	Prompt-Naive	4.7 / 97.4	6.6 / 95.1
	Prompt-Spoken	2.9 / 97.7	3.6 / 96.9

the Whisper-large-v2 model. In addition, the best system attained entity recalls of 97.7% and 96.9% for the two datasets, considerably superior to the original model’s performance of 87.3% and 79.9%, respectively.

5. Conclusion

This paper presents KWS-Whisper, an innovative ASR framework that leverages the prior knowledge of entity words to enhance their recalls. Our model consists of an OV-KWS module that exploits the hidden states of the Whisper encoder. The entities detected by this module serve as prompts for the Whisper decoder. We devise a multitask training strategy that simultaneously optimizes OV-KWS and contextual-ASR tasks. We also explore more convenient methods to apply the OV-KWS model to correct ASR errors and to prompt frozen Whisper models. Evaluated on four test sets, our method is shown to achieve substantial improvements in both MER and entity recall. In future work, we plan to enhance the efficiency of the OV-KWS module and enable it to handle a large keyword database and conduct large-scale experiments by distilling speech data from the original Whisper model, thereby boosting the performance of multitask fine-tuning in the general domain.

6. References

- [1] J. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho, and Y. Bengio, "Attention-based models for speech recognition," in *Proc. NeurIPS*, Dec. 2015, pp. 577–585.
- [2] A. Graves, "Sequence transduction with recurrent neural networks," in *Proc. ICML*, Edinburgh, Scotland, Jun. 2012.
- [3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, Jun. 2006, pp. 369–376.
- [4] A. Gourav, L. Liu, A. Gandhe, Y. Gu, G. Lan, X. Huang, S. Kalmane, G. Tiwari, D. Filimonov, A. Rastrow, A. Stolcke, and I. Bulyko, "Personalization strategies for End-to-End speech recognition systems," in *Proc. ICASSP*, 2021, pp. 7348–7352.
- [5] D. Zhao, T. N. Sainath, D. Rybach, P. Rondon, D. Bhatia, B. Li, and R. Pang, "Shallow-fusion end-to-end contextual biasing," in *Proc. Interspeech*. ISCA, sep 2019.
- [6] F.-J. Chang, J. Liu, M. Radfar, A. Mouchtaris, M. Omologo, A. Rastrow, and S. Kunzmann, "Context-aware transformer transducer for speech recognition," in *Proc. ASRU*, 2021, pp. 503–510.
- [7] Z. Chen, M. Jain, Y. Wang, M. L. Seltzer, and C. Fuegen, "Joint grapheme and phoneme embeddings for contextual end-to-end ASR," in *Proc. Interspeech*. ISCA, sep 2019.
- [8] M. Han, L. Dong, Z. Liang, M. Cai, S. Zhou, Z. Ma, and B. Xu, "Improving End-to-End contextual speech recognition with fine-grained contextual knowledge selection," in *Proc. ICASSP*, 2022, pp. 8532–8536.
- [9] T. Munkhdalai, K. C. Sim, A. Chandorkar, F. Gao, M. Chua, T. Strohmman, and F. Beaufays, "Fast contextual adaptation with neural associative memory for on-device personalized speech recognition," in *Proc. ICASSP*, 2022, pp. 6632–6636.
- [10] T. N. Sainath, R. Prabhavalkar, D. Caseiro, P. Rondon, and C. Al-lauzen, "Improving contextual biasing with text injection," in *Proc. ICASSP*, 2023, pp. 1–5.
- [11] K. M. Sathyendra, T. Muniyappa, F.-J. Chang, J. Liu, J. Su, G. P. Strimel, A. Mouchtaris, and S. Kunzmann, "Contextual adapters for personalized speech recognition in neural transducers," in *Proc. ICASSP*, 2022, pp. 8537–8541.
- [12] S. Tong, P. Harding, and S. Wiesler, "Slot-triggered contextual biasing for personalized speech recognition using neural transducers," in *Proc. ICASSP*, 2023, pp. 1–5.
- [13] G. Sun, C. Zhang, and P. C. Woodland, "Minimising biasing word errors for contextual ASR with the tree-constrained pointer generator," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 345–354, 2023.
- [14] D. Le, G. Keren, J. Chan, J. Mahadeokar, C. Fuegen, and M. L. Seltzer, "Deep shallow fusion for RNN-T personalization," in *Proc. SLT*, 2021, pp. 251–257.
- [15] Y. Xu, B. Liu, Q. Huang, X. Song, Z. Wu, S. Kang, and H. Meng, "CB-Conformer: Contextual biasing conformer for biased word recognition," in *Proc. ICASSP*, 2023, pp. 1–5.
- [16] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv preprint arXiv:2212.04356*, 2022.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30, 2017.
- [18] L. Momeni, T. Afouras, T. Stafylakis, S. Albanie, and A. Zisserman, "Seeing wake words: Audio-visual keyword spotting," *arXiv preprint arXiv:2009.01225*, 2020.
- [19] H. Shin, H. Han, D. Kim, S. Chung, and H. Kang, "Learning audio-text agreement for open-vocabulary keyword spotting," *Proc. InterSpeech*, 2022.
- [20] X. Shi, Y. Yang, Z. Li, Y. Chen, Z. Gao, and S. Zhang, "SeACo-Paraformer: A non-autoregressive ASR system with flexible and effective hotword customization ability," *arXiv preprint arXiv:2308.03266*, 2023.
- [21] Y. Li, Y. Li, M. Zhang, C. Su, J. Yu, M. Piao, X. Qiao, M. Ma, Y. Zhao, and H. Yang, "CB-whisper: Contextual biasing whisper using open-vocabulary keyword-spotting," in *Proc. LREC-COLING 2024*, Torino, Italia, May, pp. 2941–2946.
- [22] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, J. Wu, L. Zhou, S. Ren, Y. Qian, Y. Qian, J. Wu, M. Zeng, X. Yu, and F. Wei, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, p. 1505–1518, 2022.
- [23] A. Navon, A. Shamsian, N. Glazer, G. Hetz, and J. Keshet, "Open-vocabulary keyword-spotting with adaptive instance normalization," *arXiv preprint arXiv:2309.08561*, 2023.
- [24] A. Zeng, X. Liu, Z. Du, Z. Wang, H. Lai, M. Ding, Z. Yang, Y. Xu, W. Zheng, X. Xia, W. L. Tam, Z. Ma, Y. Xue, J. Zhai, W. Chen, P. Zhang, Y. Dong, and J. Tang, "Glm-130b: An open bilingual pre-trained model," *Proc. ICLR*, 2023.
- [25] W. Yu, C. Tang, G. Sun, X. Chen, T. Tan, W. Li, L. Lu, Z. Ma, and C. Zhang, "Connecting speech encoder and large language model for asr," *arXiv preprint arXiv:2309.13963*, 2023.
- [26] H. Bu, J. Du, X. Na, B. Wu, and H. Zheng, "Aishell-1: An open-source mandarin speech corpus and a speech recognition baseline," in *Proc. O-COCOSDA*, 2017, pp. 1–5.
- [27] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An asr corpus based on public domain audio books," in *Proc. ICASSP*. IEEE, 2015.
- [28] C. Li, S. Deng, Y. Wang, G. Wang, Y. Gong, C. Chen, and J. Bai, "Tals: An open-source mandarin-english code-switching corpus and a speech recognition baseline," *arXiv preprint arXiv:2206.13135*, 2022.